

Technical Handbook

CLOUD ENGINEERING

Fundamentos, Infraestrutura e Mentalidade de Sustentação

Thiago Sousa

Material pessoal de estudo e referência técnica

Este material é um documento pessoal de estudo, construído a partir da preparação de Thiago Sousa para a área de Cloud Engineering, Cloud Infrastructure e Sustentação de Ambientes Cloud.

Os conteúdos aqui reunidos combinam conhecimento efetivamente estudado e praticado com contexto complementar, claramente identificado ao longo do texto, adicionado para completar a compreensão técnica dos temas.

Versão compilada para uso pessoal, revisão e referência futura.

Contents

Introdução **7**

I Fundamentos de Computação e Infraestrutura **8**

1 Introdução à Infraestrutura Moderna **9**

1.1 Infraestrutura tradicional vs. Cloud 9

1.2 Servidores e Datacenters 9

1.3 Recursos Computacionais 9

2 Sistemas Operacionais **11**

2.1 Processos e Threads 11

2.2 Memória 11

2.3 Sistema de Arquivos e Permissões 11

2.4 Usuários, Grupos e Serviços (systemd) 12

2.5 Gerenciamento de Pacotes 12

2.6 Logs 12

2.7 Processos em Execução e Troubleshooting 13

II Redes **14**

3 Fundamentos de Redes **15**

3.1 Modelo OSI e TCP/IP 15

3.2 IP, IPv4 e IPv6 15

3.3 Máscara de Rede, CIDR e Subnet 15

3.4 Gateway e Roteamento 16

3.5 Portas, TCP e UDP 16

3.6 HTTP, HTTPS e TLS 16

3.7 DHCP 16

3.8 NAT 17

3.9 Firewall e Security Groups 17

4 DNS **18**

4.1 Resolução de Nomes 18
4.2 Tipos de Registro 18
4.3 Diagnóstico de Problemas de DNS 19

III Virtualização **20**

5 Virtualização **21**
5.1 Máquinas Virtuais e Hypervisor 21
5.2 Virtualização de CPU, Memória e Armazenamento 21
5.3 Redes Virtuais 21
5.4 Snapshots, Templates e Provisionamento 21
5.5 Virtualização vs. Containers 22
5.6 Relação com Cloud Computing 22

IV Cloud Computing **23**

6 Fundamentos de Cloud **24**
6.1 O que é Cloud Computing 24
6.2 IaaS, PaaS e SaaS 24
6.3 Elasticidade e Escalabilidade 24
6.4 Alta Disponibilidade, Regiões e Zonas 25
6.5 Fault Tolerance e Pay-as-you-go 25
6.6 Shared Responsibility Model 25

V AWS **26**

7 AWS Fundamentals **27**
7.1 Contas, Regiões e Availability Zones 27
7.2 IAM: Identity and Access Management 27
7.3 EC2: Elastic Compute Cloud 28
7.4 EBS: Elastic Block Store 28

8 CloudWatch e SSM **29**

VI VPC e Networking na Cloud **30**

9 Arquitetura de uma VPC **31**

9.1 Fluxo Geral . . . 31
9.2 Public Subnet vs. Private Subnet . . . 31
9.3 Route Tables . . . 32
9.4 Isolamento entre Subnets . . . 32
9.5 Exemplo Prático: Configurando Security Groups em Camadas . . . 32
9.6 Load Balancer . . . 32

VII Containers **34**

10 Docker **35**

10.1 Conceitos Fundamentais . . . 35
10.2 Relação com Virtualização e Cloud . . . 35
10.3 Dockerfile e Comandos Básicos . . . 35

VIII Orquestração **37**

11 Kubernetes **38**

11.1 Conceitos Fundamentais . . . 38
11.2 Scaling, Scheduling e Health Checks . . . 38

IX Infrastructure as Code **39**

12 Terraform **40**

12.1 Por que IaC é importante . . . 40
12.2 Conceitos do Terraform . . . 40
12.3 Drift e Organização em Módulos . . . 41

X Scripting e Automação **42**

13 Bash **43**

13.1 Fundamentos . . . 43
13.2 Redirecionamento e Pipes . . . 43
13.3 Ferramentas de Texto: grep, awk, sed, cut, sort, xargs . . . 43

13.4 Cron 44
13.5 Script Real: Verificação de Serviço com Alerta 44

14 Python para Infraestrutura **45**
14.1 Bibliotecas Relevantes 45
14.2 Script Real: Monitoramento de Disco 45
14.3 Extensão Natural: psutil e requests 46

XI Observabilidade **47**

15 Monitoramento e Observabilidade **48**
15.1 Métricas, Logs, Traces e Eventos 48
15.2 Monitoramento vs. Observabilidade vs. Troubleshooting 48

XII Sustentação e Troubleshooting **49**

16 Mentalidade de Sustentação **50**
16.1 Metodologia de Investigação 50
16.2 Cenário 1: Servidor Inacessível 51
16.3 Cenário 2: Aplicação Lenta 51
16.4 Cenário 3: Aplicação Não Consegue Acessar o Banco 51
16.5 Cenário 4: Servidor Sem Espaço 52

XIII Projetos e Laboratórios **53**

17 Laboratórios Realizados **54**
17.1 Laboratório 1 — Script de Monitoramento de Disco (Python) 54
17.2 Laboratório 2 — Script de Verificação de Serviço (Bash) 55
17.3 Laboratório 3 — Arquitetura Conceitual de VPC 55

XIV Arquitetura de Ponta a Ponta **57**

18 Conectando Todos os Conceitos **58**

XV Entrevista de Cloud Engineering **59**

19 Perguntas e Respostas por Nível **60**

19.1 Nível Básico — Fundamentos 60
19.2 Nível Intermediário — Troubleshooting 61
19.3 Nível Avançado — Arquitetura e Incidêntes 62

XVI Glossário **63**

Glossário Técnico **64**

Introdução

Este handbook nasceu de um processo simples: curiosidade encontrando um problema real e não soltando mais.

Minha trajetória começou no desenvolvimento de software — construíndo APIs, integrando sistemas, resolvendo regras de negócio. Foi lidando com um problema aparentemente pequeno, containers Docker que simplesmente não se comunicavam por causa de um comportamento específico do sistema operacional, que percebi que o que mais me atraía não era só construir a aplicação, mas entender o ambiente em que ela roda: o sistema operacional por baixo, a rede que conecta as peças, a infraestrutura que sustenta tudo funcionando.

Esse material reúne os fundamentos que estudei nessa jornada em direção à Cloud Engineering: sistemas operacionais, redes, virtualização, computação em nuvem, arquitetura de VPC, containers, automação, observabilidade e a mentalidade de sustentação de ambientes críticos.

Não pretendo aparentar domínio completo de todos os temas aqui presentes — isso seria desonesto com o estágio real da minha formação. O objetivo deste documento é outro: registrar de forma organizada e tecnicamente rigorosa aquilo que venho estudando, de modo que sirva tanto como material de revisão pessoal quanto como evidência de uma forma de pensar que estou desenvolvendo.

ATENÇÃO

Sempre que um conceito exigir conhecimento complementar àquele que venho estudando e praticando diretamente, isso será identificado explicitamente em caixas de **Contexto Complementar**, para manter clareza entre o que foi vivenciado na prática e o que é apresentado para completar a compreensão técnica do assunto.

A jornada documentada aqui segue uma progressão lógica:



E, a partir daí, para automação, observabilidade e sustentação — as camadas que transformam infraestrutura funcionando em infraestrutura confiável.

"Um bom profissional de infraestrutura não apenas sabe executar comandos. Ele entende o sistema, formula hipóteses, coleta evidências, identifica a causa raiz e toma decisões baseadas em dados."

I

Fundamentos de Computação e Infraestrutura

CAPÍTULO 1

Introdução à Infraestrutura Moderna

Antes de falar de nuvem, vale entender o que ela substitui e o que preserva. **Infraestrutura** é o conjunto de recursos físicos e lógicos — servidores, rede, armazenamento — que sustenta uma aplicação rodando e disponível para quem precisa dela.

1.1 Infraestrutura tradicional vs. Cloud

Na infraestrutura tradicional (*on-premises*), a empresa compra, instala e mantém fisicamente seus próprios servidores num datacenter próprio ou alugado. Isso significa: investimento alto adiantado, tempo de provisionamento medido em semanas, e capacidade ociosa (você compra pensando no pico de uso, e o resto do tempo esse hardware fica subutilizado).

Na Cloud, um provedor (AWS, Azure, GCP, ou plataformas nacionais como a Locaweb Cloud) mantém datacenters massivos e aluga capacidade computacional sob demanda. Isso inverte a lógica: você paga pelo que usa (*pay-as-you-go*), provisiona recursos em minutos, e escala para cima ou para baixo conforme a necessidade real.

Aspecto	**Tradicional**	**Cloud**
Investimento	Alto, adiantado (CAPEX)	Sob demanda (OPEX)
Tempo de provisionamento	Dias a semanas	Minutos
Escalabilidade	Limitada ao hardware comprado	Elástica
Manutenção física	Responsabilidade da empresa	Responsabilidade do provedor
Capacidade ociosa	Comum (dimensionado para o pico)	Minimizada (paga-se o uso real)

1.2 Servidores e Datacenters

Um **servidor** é, em essência, um computador dedicado a atender requisições de outros computadores (clientes). Um **datacenter** é uma instalação física que abriga milhares desses servidores, com energia redundante, refrigeração, conectividade de rede de alta capacidade e segurança física.

1.3 Recursos Computacionais

Todo workload — seja uma aplicação web, um banco de dados ou um script de automação — consome quatro tipos fundamentais de recurso:

- **CPU**: capacidade de processamento, medida em núcleos (*cores*) e frequência.
- **Memória (RAM)**: espaço voltátil onde dados em uso ficam disponíveis rapidamente para a CPU.

- **Armazenamento (Storage)**: espaço persistente em disco, onde dados sobrevivem a reinicializações.
- **Rede**: capacidade de comunicação entre sistemas, medida em largura de banda e latência.

Workload é o termo usado para descrever qualquer carga de trabalho computacional — uma aplicação, um banco de dados, um job de processamento — rodando sobre esses recursos.

CAPÍTULO 2

Sistemas Operacionais

O sistema operacional é a camada que faz a ponte entre o hardware físico e as aplicações. Em ambientes de sustentação e Cloud, o Linux é o padrão de mercado — a imensa maioria dos servidores em nuvem roda alguma distribuição Linux (Ubuntu, Amazon Linux, Debian, entre outras).

2.1 Processos e Threads

Um **processo** é uma instância de um programa em execução, com seu próprio espaço de memória isolado. Uma **thread** é uma linha de execução dentro de um processo — várias threads podem rodar dentro do mesmo processo, compartilhando memória entre si, o que as torna mais leves que processos completos, porém mais suscetíveis a condições de corrida se mal sincronizadas.

2.2 Memória

O kernel do Linux gerencia a memória disponível entre processos, incluindo o uso de **swap** — espaço em disco usado como extensão da RAM quando ela se esgota. Uso excessivo de swap é sinal de alerta: significa que o sistema está sob pressão de memória e a performance provavelmente está degradada, já que disco é ordens de magnitude mais lento que RAM.

2.3 Sistema de Arquivos e Permissões

A estrutura de diretórios do Linux segue um padrão (*Filesystem Hierarchy Standard*):

Caminho	Finalidade
`/`	Raiz do sistema de arquivos
`/home`	Diretórios pessoais dos usuários
`/etc`	Arquivos de configuração do sistema
`/var/log`	Logs do sistema e de serviços
`/bin`, `/usr/bin`	Binários executáveis
`/tmp`	Arquivos temporários

Permissões seguem o modelo `rwx` (leitura, escrita, execução), aplicado em três níveis: dono do arquivo, grupo, e demais usuários.

Listing 2.1: Permissões: leitura de um exemplo

```
$ ls -l script.sh
-rwxr-xr-- 1 thiago devops 512 Set  6 10:00 script.sh
```

Nesse exemplo: o dono (`thiago`) tem leitura, escrita e execução; o grupo (`devops`) tem leitura e execução; outros usuários têm apenas leitura.

Listing 2.2: Comandos de permissão mais usados

```
chmod +x script.sh          # adiciona permissao de execucao
chmod 755 script.sh         # define rwx para dono, r-x para grupo/outros
chown usuario:grupo arq     # muda o dono e o grupo do arquivo
```

2.4 Usuários, Grupos e Serviços (systemd)

O Linux moderno usa o **systemd** como sistema de inicialização e gerenciador de serviços. Ele controla o ciclo de vida de serviços (*daemons*) como servidores web, bancos de dados e agentes de monitoramento.

Listing 2.3: Gerenciamento de serviços com systemctl

```
systemctl status nginx       # ver status atual do servico
systemctl start nginx        # iniciar o servico
systemctl stop nginx         # parar o servico
systemctl restart nginx      # reiniciar o servico
systemctl enable nginx       # habilitar inicio automatico no boot
systemctl is-active --quiet nginx   # retorna codigo de saida (uso em
    scripts)
```

2.5 Gerenciamento de Pacotes

Listing 2.4: Gerenciadores de pacote comuns

```
# Debian/Ubuntu
apt update && apt install nome-pacote

# Amazon Linux/RHEL/CentOS
yum install nome-pacote
dnf install nome-pacote
```

2.6 Logs

Logs são a principal fonte de evidência em troubleshooting. A maioria dos serviços grava logs em `/var/log`, e o próprio systemd centraliza logs via `journalctl`.

Listing 2.5: Investigando logs

```
tail -f /var/log/syslog          # acompanhar log em tempo real
journalctl -u nginx -f           # logs de um servico especifico, em
    tempo real
journalctl --since "10 min ago"  # logs recentes
grep -i "error" /var/log/app.log # filtrar linhas com "error"
```

2.7 Processos em Execução e Troubleshooting

Listing 2.6: Comandos essenciais de diagnóstico

```
ps aux                  # lista todos os processos em execucao
top                     # visao em tempo real de CPU/memoria por processo
htop                    # versao mais amigavel do top (se instalado)
df -h                   # espaco em disco, formato legivel
free -h                 # uso de memoria RAM e swap
netstat -tulnp          # portas abertas e processos associados
ss -tulnp               # alternativa moderna ao netstat
kill -9 PID             # forca o encerramento de um processo (ultimo recurso)
crontab -l              # lista tarefas agendadas do usuario atual
```

ATENÇÃO

`kill -9` envia o sinal `SIGKILL`, que encerra o processo imediatamente, sem chance de limpeza (fechar arquivos, liberar recursos). Prefira sempre um `systemctl restart` ou um `kill` padrão (`SIGTERM`) primeiro, e reserve o `-9` para quando o processo realmente não responde a mais nada.

Um conceito importante para entrevistas técnicas é a diferença entre um **processo zumbi** e um **processo travado**: um zumbi já terminou sua execução, mas seu processo pai ainda não coletou seu código de saída (fica "pendurado" na tabela de processos, sem consumir CPU de fato); um processo travado continua rodando, geralmente preso em algum loop ou aguardando um recurso que nunca chega, e pode consumir CPU ativamente.

II

Redes

CAPÍTULO 3

Fundamentos de Redes

3.1 Modelo OSI e TCP/IP

O modelo OSI descreve comunicação de rede em 7 camadas teóricas. Na prática do dia a dia de infraestrutura, o modelo **TCP/IP**, mais simplificado, é o que realmente orienta o raciocínio:

Camada TCP/IP	Exemplos
Aplicação	HTTP, HTTPS, DNS, SSH
Transporte	TCP, UDP
Internet	IP, roteamento
Enlace/Física	Ethernet, switches, Wi-Fi

CONTEXTO COMPLEMENTAR

O modelo OSI (7 camadas: Física, Enlace, Rede, Transporte, Sessão, Apresentação, Aplicação) é mais usado como referência acadêmica e didática. No dia a dia de sustentação, o modelo TCP/IP de 4 camadas é o que efetivamente orienta o diagnóstico de problemas.

3.2 IP, IPv4 e IPv6

Um endereço **IP** identifica unicamente um dispositivo numa rede. O **IPv4** usa 32 bits, representado em 4 octetos decimais (ex: `192.168.1.10`), oferecendo cerca de 4,3 bilhões de endereços — número já insuficiente para a quantidade de dispositivos conectados no mundo. O **IPv6** usa 128 bits, resolvendo esse limite com um espaço de endereços praticamente inesgotável, representado em blocos hexadecimais (ex: `2001:0db8::1`).

3.3 Máscara de Rede, CIDR e Subnet

A notação **CIDR** (*Classless Inter-Domain Routing*) representa um bloco de endereços IP na forma `IP/prefixo`, onde o número após a barra indica quantos bits representam a parte de **rede** do endereço — o restante dos bits define o espaço de endereços disponíveis dentro daquela rede.

Listing 3.1: Exemplos de blocos CIDR

```
10.0.0.0/16     -> ate 65.536 enderecos (parte da VPC inteira)
10.0.1.0/24     -> ate 256 enderecos (uma subnet)
192.168.1.0/24 -> ate 256 enderecos (rede domestica tipica)
```

Uma **subnet** é uma subdivisão lógica de uma rede maior, usada para organizar recursos em grupos com propósitos distintos (ex: uma subnet para servidores web, outra para banco de dados) e para aplicar controles de acesso diferentes a cada grupo.

CONTEXTO COMPLEMENTAR

Em provedores de nuvem, é comum cada subnet estar associada a uma única *Availability Zone* (zona de disponibilidade) — uma subnet não "atravessa" zonas.

3.4 Gateway e Roteamento

Um **gateway** é o ponto de saída de uma rede local para outras redes (incluindo a internet). Uma **tabela de rotas** (*route table*) define, para cada faixa de destino, por qual gateway o tráfego deve seguir.

3.5 Portas, TCP e UDP

Enquanto o IP identifica a **máquina**, a **porta** identifica o **serviço/processo** específico dentro dela.

Porta	Serviço
22	SSH
80	HTTP
443	HTTPS
53	DNS
3306	MySQL
5432	PostgreSQL

TCP é orientado a conexão: antes de trocar dados, faz um *handshake* de três vias (`SYN` → `SYN-ACK` → `ACK`), garantindo entrega confiável e ordenada. É usado onde perder dado não é aceitável: HTTP, SSH, transferência de arquivos, bancos de dados.

UDP não estabelece conexão prévia — simplesmente envia pacotes, sem garantia de entrega ou ordem. Isso o torna mais rápido, e por isso é usado em streaming de vídeo, chamadas de voz e consultas DNS simples, onde perder um pacote ocasional é preferível a esperar reenvio.

3.6 HTTP, HTTPS e TLS

HTTP é o protocolo de comunicação da web, baseado em requisições e respostas em texto claro. **HTTPS** é o mesmo protocolo, envolto numa camada de criptografia via **TLS** (*Transport Layer Security*), garantindo que o conteúdo trafegado não possa ser lido ou alterado por terceiros no caminho.

3.7 DHCP

DHCP (*Dynamic Host Configuration Protocol*) atribui automaticamente endereços IP a dispositivos que entram numa rede, eliminando a necessidade de configuração manual de IP em cada máquina.

3.8 NAT

NAT (*Network Address Translation*) traduz endereços IP privados para um endereço público compartilhado, permitindo que máquinas numa rede interna acessem a internet sem cada uma precisar de um IP público próprio. Em Cloud, o **NAT Gateway** cumpre esse papel para recursos em subnets privadas: eles conseguem iniciar conexões para fora, mas não recebem conexões iniciadas de fora.

Diagrama — NAT Gateway

```
          INTERNET
             |
       NAT GATEWAY
             |
      REDE PRIVADA
        /       \
     VM 1       VM 2

Um unico NAT Gateway atende multiplas VMs privadas.
As VMs iniciam conexao para fora; a internet NAO
consegue iniciar conexao para dentro.
```

3.9 Firewall e Security Groups

Um **firewall** filtra tráfego de rede com base em regras (origem, destino, porta, protocolo). Em Cloud, o **Security Group** funciona como um firewall associado diretamente a um recurso (como uma instância), controlando quais conexões de entrada e saída são permitidas para aquele recurso especificamente — é diferente de uma *subnet*, que apenas define onde o recurso está na rede e quais rotas ele possui.

ATENÇÃO

Uma instância numa subnet **pública** ainda pode ter todo o tráfego de entrada bloqueado se o Security Group associado a ela negar essas conexões. Subnet e Security Group atuam em camadas independentes: a subnet permite a rota, mas o Security Group decide se a conexão realmente passa.

CAPÍTULO 4

DNS

4.1 Resolução de Nomes

DNS (*Domain Name System*) traduz nomes de domínio legíveis por humanos (`exemplo.com.br`) em endereços IP que as máquinas de fato usam para se comunicar.

Diagrama — Fluxo de resolução DNS

```
Seu computador
     |
Resolver Recursivo (ISP / DNS local)
     |
Root Servers  (.)
     |
TLD Servers   (.com.br)
     |
Servidor Autoritativo do dominio
     |
IP retornado
```

4.2 Tipos de Registro

Registro	Função
A	Aponta o domínio para um endereço IPv4
AAAA	Aponta o domínio para um endereço IPv6
CNAME	Cria um alias, apontando para outro nome de domínio
MX	Define o(s) servidor(es) responsável(is) por e-mail do domínio
TXT	Armazena texto arbitrário, usado para validações (SPF, verificação de propriedade)
NS	Indica quais servidores são autoritativos para o domínio

TTL (*Time To Live*) define por quanto tempo uma resposta DNS pode ficar em cache antes de ser consultada novamente — TTL baixo permite mudanças rápidas de propagação, à custa de mais consultas; TTL alto reduz carga nos servidores DNS, à custa de propagação mais lenta em caso de mudança.

Recursive resolver é o servidor que faz todo o trabalho de navegar pela hierarquia DNS em nome do cliente; **authoritative DNS** é o servidor que efetivamente detém e responde pelo registro final de um domínio específico.

4.3 Diagnóstico de Problemas de DNS

Listing 4.1: Ferramentas de diagnóstico de rede e DNS

```
dig exemplo.com                # consulta DNS detalhada
nslookup exemplo.com           # consulta DNS simples
host exemplo.com               # consulta DNS resumida
curl -I https://exemplo.com    # verifica resposta HTTP (cabecalhos)
ping exemplo.com               # testa conectividade basica e latencia
traceroute exemplo.com         # mostra o caminho (saltos) ate o destino
ss -tulnp                      # portas abertas localmente
ip a                           # mostra interfaces de rede e IPs locais
```

Problema comum: domínio não resolve

Sintoma: `curl` retorna erro de resolução de nome.
Possíveis causas: registro DNS inexistente ou incorreto; TTL expirado sem propagação correta; resolver local com cache corrompido; firewall bloqueando porta 53.
Comandos: `dig dominio.com`, `dig dominio.com @8.8.8.8` (testando um resolver externo diretamente).
Interpretação: se o `dig` contra um resolver externo funciona mas o local não, o problema está no resolver/cache local, não no registro em si.

III

Virtualização

CAPÍTULO 5

Virtualização

5.1 Máquinas Virtuais e Hypervisor

Virtualização é a técnica que permite executar múltiplos sistemas isolados (**máquinas virtuais**) sobre um único hardware físico. Cada VM tem sua própria CPU virtual, memória, disco e sistema operacional completo, como se fosse uma máquina física independente.

Quem gerencia essas VMs é o **hypervisor**:

Tipo	Característica
Tipo 1 (bare-metal)	Roda diretamente sobre o hardware, sem sistema operacional hospedeiro. Usado em produção/datacenter. Exemplos: VMware ESXi, XCP-ng, Microsoft Hyper-V.
Tipo 2 (hosted)	Roda como uma aplicação sobre um sistema operacional já instalado. Usado em ambientes de desenvolvimento/teste local. Exemplo: VirtualBox.

CONTEXTO COMPLEMENTAR

XCP-ng é um hypervisor *open-source* baseado no projeto Xen, funcionando como alternativa gratuita ao VMware ESXi — é citado como diferencial em vagas de sustentação de infraestrutura, ao lado do próprio VMware.

5.2 Virtualização de CPU, Memória e Armazenamento

O hypervisor abstrai os recursos físicos e os distribui entre as VMs: CPU virtual (*vCPU*) mapeia para tempo de execução na CPU física; memória é alocada em blocos isolados por VM; e armazenamento é apresentado como discos virtuais, que na prática são arquivos ou volumes no storage físico subjacente.

5.3 Redes Virtuais

Assim como recursos de computação, a rede também é virtualizada: um **switch virtual** conecta as VMs entre si e ao mundo externo, permitindo isolamento lógico (VLANs virtuais) sem depender de cabeamento físico separado.

5.4 Snapshots, Templates e Provisionamento

Um **snapshot** é um ponto de restauração do estado de um disco/VM em determinado momento — permite reverter alterações problemáticas rapidamente.

Diagrama — Ciclo de um snapshot

```
VM funcionando normalmente
        |
   cria snapshot
        |
  aplica alteracao arriscada
        |
   algo da errado
        |
  restaura o snapshot
        |
VM volta ao estado anterior
```

Um **template** é uma imagem base pré-configurada, usada como ponto de partida para criar novas VMs rapidamente, sem repetir manualmente toda a instalação e configuração inicial.

5.5 Virtualização vs. Containers

É comum confundir VM com container, mas a diferença é fundamental:

	Máquina Virtual	**Container**
Isolamento	Hardware virtualizado completo	Processos isolados, kernel compartilhado
Sistema operacional	Um SO completo por VM	Compartilha o kernel do host
Peso	Pesado (GBs)	Leve (MBs)
Tempo de inicialização	Minutos	Segundos
Isolamento de segurança	Mais forte (físico)	Mais fraco (lógico)

5.6 Relação com Cloud Computing

A Cloud, em sua essência, é virtualização em escala massiva, orquestrada por software: quando você cria uma instância EC2 na AWS, por exemplo, você está solicitando uma VM provisionada automaticamente sobre a infraestrutura física compartilhada do provedor.

PERGUNTA DE REVISÃO

Por que uma empresa que já usa containers ainda investe em VMs?
Porque VMs oferecem isolamento mais forte (útil para multi-tenancy e segurança), enquanto containers priorizam eficiência e velocidade. Muitas arquiteturas modernas usam containers rodando dentro *de VMs, combinando os dois níveis de isolamento.*

IV

Cloud Computing

CAPÍTULO 6

Fundamentos de Cloud

6.1 O que é Cloud Computing

Cloud Computing é a entrega de recursos computacionais (processamento, armazenamento, rede, bancos de dados, entre outros) sob demanda, via internet, com cobrança baseada em uso, sem que o cliente precise possuir ou administrar a infraestrutura física subjacente.

6.2 IaaS, PaaS e SaaS

Modelo	O que o provedor entrega	O que você gerencia
IaaS	Infraestrutura bruta (VMs, rede, storage)	Sistema operacional, runtime, aplicação
PaaS	Plataforma pronta para rodar código	Apenas sua aplicação e dados
SaaS	Software completo, pronto para uso final	Apenas configuração de uso

Exemplos: EC2 (AWS) é IaaS; um serviço gerenciado que só pede seu código, como Heroku ou AWS Elastic Beanstalk, é PaaS; Gmail ou Salesforce são SaaS.

6.3 Elasticidade e Escalabilidade

Elasticidade é a capacidade de um ambiente se ajustar automaticamente — para cima ou para baixo — conforme a demanda muda em tempo real. **Escalabilidade** é a capacidade do sistema de crescer para suportar mais carga, e se divide em duas estratégias:

- **Vertical**: aumentar os recursos de uma única máquina (mais CPU, mais RAM). Tem um teto físico — na prática, a partir de certo ponto (uma referência comum é por volta de 64GB de RAM), o custo e a complexidade de continuar escalando verticalmente superam os benefícios.
- **Horizontal**: adicionar mais máquinas, distribuindo a carga entre elas (geralmente via um *Load Balancer*). Não tem o mesmo teto físico, e ainda traz redundância: se uma máquina falha, as demais continuam atendendo.

PERGUNTA DE REVISÃO

Quando escalar verticalmente ainda faz mais sentido que horizontalmente?
Em cargas que são difíceis de distribuir entre máquinas, como um banco de dados relacional único com muitas transações dependentes entre si — dividir isso entre várias máquinas introduz complexidade de sincronização que pode não compensar.

6.4 Alta Disponibilidade, Regiões e Zonas

Região é uma área geográfica onde o provedor mantém infraestrutura (ex: São Paulo, Virgínia do Norte). Cada região é dividida em **zonas de disponibilidade** (*Availability Zones*) — datacenters fisicamente separados, com energia e conectividade independentes, mas interligados por rede de baixa latência.

Alta disponibilidade é alcançada distribuindo recursos entre múltiplas zonas: se uma zona inteira falhar, o serviço continua rodando na outra.

Diagrama — Alta disponibilidade com múltiplas zonas

```
              INTERNET
                 |
           LOAD BALANCER
              /      \
     Zona A            Zona B
       VM1               VM2

Se a Zona A cair por completo, o Load Balancer
direciona todo o trafego para a Zona B.
```

6.5 Fault Tolerance e Pay-as-you-go

Fault tolerance (tolerância a falhas) é a capacidade do sistema continuar operando — ainda que de forma degradada — mesmo diante da falha de um componente. **Pay-as-you-go** é o modelo de cobrança por uso real, sem contratos fixos de capacidade mínima obrigatória.

6.6 Shared Responsibility Model

O **modelo de responsabilidade compartilhada** define os limites entre o que o provedor de nuvem garante e o que é responsabilidade do cliente:

Responsável	Cobre
Provedor (AWS/Azure/etc.)	Segurança física dos datacenters, hardware, virtualização, rede global
Cliente	Configuração de segurança (Security Groups, IAM), sistema operacional (em IaaS), dados, aplicação

ATENÇÃO

Um erro comum de quem está começando é assumir que "está na nuvem" significa que a segurança é problema exclusivo do provedor. Na prática, a maioria dos incidentes de segurança em Cloud vem de configuração incorreta do lado do cliente — um Security Group aberto demais, credenciais expostas, um bucket de armazenamento público sem necessidade.



AWS

CAPÍTULO 7

AWS Fundamentals

Esta seção organiza os principais serviços e conceitos da AWS estudados, sempre seguindo a progressão: **o que é → por que existe → como funciona → quando usar → problemas comuns → como diagnosticar**.

7.1 Contas, Regiões e Availability Zones

Uma conta AWS é a unidade administrativa e de cobrança. Dentro dela, recursos são criados em **regiões** específicas (ex: `sa-east-1` para São Paulo), e cada região contém várias **Availability Zones** isoladas entre si fisicamente.

7.2 IAM: Identity and Access Management

O que é: serviço de gerenciamento de identidade e permissões da AWS. **Por que existe:** para controlar precisamente quem (pessoa ou sistema) pode fazer o quê, sobre quais recursos. **Como funciona:** através de *users*, *groups*, *roles* e *policies*.

Elemento	Função
User	Identidade de uma pessoa ou aplicação específica
Group	Coleção de users, para aplicar permissões em lote
Role	Identidade temporária, assumível por serviços ou usuários, sem credenciais fixas
Policy	Documento JSON que define quais ações são permitidas ou negadas
MFA	Autenticação multifator, camada extra de segurança no login

Quando usar Role em vez de credenciais fixas de User: sempre que possível — roles geram credenciais temporárias, reduzindo o risco de uma chave de acesso vazada continuar válida indefinidamente.

Listing 7.1: Exemplo simplificado de IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::meu-bucket/*"
    }
  ]
}
```

Essa policy permite apenas a ação de **leitura** (`GetObject`) sobre objetos dentro de um bucket S3 específico — exemplo do princípio de **menor privilégio**: conceder exatamente o necessário, nada além disso.

Problemas comuns: permissões excessivamente amplas (`"Action": "*"`); usuários com credenciais fixas antigas nunca rotacionadas; falta de MFA em contas privilegiadas. **Como diagnosticar:** revisar policies anexadas a um user/role via console ou CLI, verificar o *IAM Access Analyzer* para identificar acessos públicos ou entre contas não intencionais.

7.3 EC2: Elastic Compute Cloud

O que é: serviço de máquinas virtuais sob demanda da AWS — o equivalente direto a uma VM tradicional. **Por que existe:** fornecer capacidade de processamento configurável, sem necessidade de hardware próprio.

Conceito	Descrição
AMI	Imagem base (sistema operacional + configurações) usada para lançar a instância
Instance Type	Combinação de CPU, RAM e rede disponível (ex: t3.micro, m5.large)
EBS	Volume de armazenamento persistente anexado à instância
Key Pair	Par de chaves SSH para acesso seguro à instância
User Data	Script executado automaticamente na primeira inicialização

7.4 EBS: Elastic Block Store

O que é: armazenamento em bloco, separado do ciclo de vida da instância EC2 — é possível desanexar um volume EBS de uma instância e anexar em outra, preservando os dados.

ATENÇÃO

Sistema operacional é diferente de dados da aplicação. Uma boa prática arquitetural é separar o **root disk** (onde vive o SO) do **data disk** (onde vivem os dados persistentes da aplicação), facilitando backup, migração e resposta a incidentes sem depender do estado exato do disco de sistema.

Um indicador de performance de storage relevante é o **IOPS** (*Input/Output Operations Per Second*) — quantas operações de leitura/escrita o armazenamento consegue processar por segundo. Cargas de trabalho com muitas transações pequenas e frequentes (bancos de dados, por exemplo) costumam exigir volumes com IOPS mais altos.

CAPÍTULO 8

CloudWatch e SSM

CloudWatch é o serviço de monitoramento e logs da AWS — coleta métricas (CPU, memória via agente, tráfego de rede), permite configurar alarmes, e centraliza logs de aplicações e serviços. É a peça central para conectar infraestrutura com **sustentação e reporte de incidentes**: sem métricas e logs centralizados, diagnosticar um problema em produção vira adivinhação.

SSM (*Systems Manager*) permite gerenciar instâncias EC2 remotamente — executar comandos, aplicar patches, ou até acessar um terminal — sem precisar de acesso SSH direto exposto à internet, o que reduz superfície de ataque.

VI

VPC e Networking na Cloud

CAPÍTULO 9

Arquitetura de uma VPC

9.1 Fluxo Geral

Diagrama — Camadas de uma VPC

```
              INTERNET
                 |
          INTERNET GATEWAY
                 |
=====================================
                VPC (10.0.0.0/16)
-------------------------------------
  Subnet Publica       10.0.1.0/24
     Load Balancer
-------------------------------------
  Subnet Privada       10.0.2.0/24
     EC2 (aplicacao)
-------------------------------------
  Subnet Privada       10.0.3.0/24
     RDS (banco de dados)
=====================================
                |
        ROUTE TABLES  +  SECURITY GROUPS
```

9.2 Public Subnet vs. Private Subnet

	Subnet Pública	**Subnet Privada**
Rota para internet	Via Internet Gateway (entrada e saída)	Via NAT Gateway (apenas saída)
Recebe conexão externa	Sim, se o Security Group permitir	Não, nunca diretamente
Uso típico	Load Balancer, bastion host	Aplicação, banco de dados

ATENÇÃO

Uma subnet privada não significa ausência total de internet — significa ausência de conexões **iniciadas de fora para dentro**. Um recurso numa subnet privada ainda pode acessar a internet para saída (baixar atualizações, chamar uma API externa) através de um NAT Gateway.

9.3 Route Tables

Cada subnet está associada a uma **tabela de rotas**, que determina para onde o tráfego de saída é direcionado com base no endereço de destino. Uma subnet é considerada "pública" justamente porque sua tabela de rotas contém uma rota para o Internet Gateway.

9.4 Isolamento entre Subnets

Mesmo dentro da mesma VPC, o tráfego entre subnets diferentes pode ser controlado e restringido via Security Groups e, de forma complementar, via **Network ACLs** — regras de firewall aplicadas no nível da subnet como um todo, e não de um recurso individual.

CONTEXTO COMPLEMENTAR

Network ACLs (NACLs) operam de forma *stateless* (cada direção de tráfego precisa de regra explícita), enquanto Security Groups são *stateful* (uma resposta a uma conexão permitida é automaticamente aceita de volta). Essa diferença costuma aparecer em entrevistas mais avançadas de rede em Cloud.

9.5 Exemplo Prático: Configurando Security Groups em Camadas

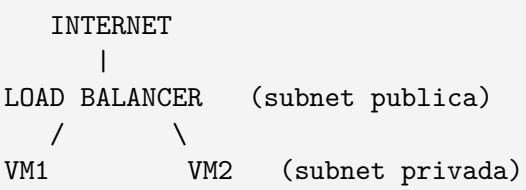
Aplicação web + banco de dados

Dois Security Groups separados, seguindo o princípio de menor privilégio:
SG da aplicação web: libera portas 80/443 de qualquer origem (`0.0.0.0/0`); porta 22 (SSH) liberada apenas para o IP do administrador ou uma VPN.
SG do banco de dados: não libera nenhuma porta para a internet. A única origem permitida na porta do banco (ex: 5432 para PostgreSQL) é o próprio Security Group da aplicação web — garantindo que só a aplicação consiga falar com o banco, mesmo que alguém descubra o IP dele.

9.6 Load Balancer

O que é: recurso que distribui requisições de entrada entre múltiplas instâncias, viabilizando escalabilidade horizontal. **Onde fica:** tipicamente numa subnet pública, já que é o ponto de entrada do tráfego vindo da internet — as instâncias que ele distribui podem (e devem, por segurança) ficar em subnets privadas.

Diagrama — Load Balancer distribuindo tráfego



Como garante alta disponibilidade: através de **health checks** periódicos — se uma instância

para de responder, o Load Balancer deixa de enviar tráfego para ela automaticamente, redirecionando tudo para as instâncias saudáveis restantes, até que ela volte a responder normalmente.

VII

Containers

CAPÍTULO 10

Docker

10.1 Conceitos Fundamentais

Termo	Definição
Container	Processo isolado, empacotado com tudo que precisa para rodar (código, dependências, configuração), compartilhando o kernel do host
Image	Modelo imutável a partir do qual containers são criados
Dockerfile	Arquivo de instruções que define como construir uma image
Registry	Repositório onde images ficam armazenadas e versões são publicadas
Docker Hub	Registry público mais popular
Volume	Mecanismo de persistência de dados fora do ciclo de vida do container
Network	Camada de rede virtual que conecta containers entre si

10.2 Relação com Virtualização e Cloud

Containers resolvem um problema prático que VMs resolvem de forma mais pesada: empacotar uma aplicação com tudo que ela precisa para rodar de forma consistente, independentemente de onde é executada. Em Cloud, é comum rodar containers *dentro* de VMs (instâncias EC2, por exemplo), combinando o isolamento físico da VM com a leveza e portabilidade do container.

ATENÇÃO

Um comportamento específico do sistema operacional hospedeiro pode interferir na comunicação entre containers mesmo quando a configuração de rede parece correta à primeira vista — um exemplo real vivido: dois containers (uma API principal e uma API mock) que simplesmente não se comunicavam devido a um comportamento específico do Ubuntu na camada de rede do Docker. Isso reforça uma lição central de troubleshooting: ás vezes o problema não está na aplicação, está no ambiente em que ela roda.

10.3 Dockerfile e Comandos Básicos

Listing 10.1: Exemplo simples de Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

```

```
EXPOSE 5000
CMD ["python", "app.py"]
```

Listing 10.2: Comandos essenciais do Docker

```
docker build -t minha-app .           # constroi a imagem a partir do Dockerfile
docker run -p 5000:5000 minha-app     # executa um container, mapeando portas
docker ps                             # lista containers em execucao
docker logs -f nome_container         # acompanha logs de um container
docker exec -it nome_container bash   # abre um terminal dentro do container
docker compose up                     # sobe multiplos servicos definidos em compose.yml
```

CONTEXTO COMPLEMENTAR

Variáveis de ambiente (`-e VAR=valor` no `docker run`, ou a seção `environment` no Docker Compose) são a forma padrão de injetar configuração — como credenciais de banco de dados — sem hardcodar valores dentro da imagem.

VIII

Orquestração

CAPÍTULO 11

Kubernetes

CONTEXTO COMPLEMENTAR

Os fundamentos de Kubernetes apresentados nesta seção são **contexto complementar**, incorporados para completar a visão de ponta a ponta de uma arquitetura moderna de containers em escala — não representam prática direta ainda realizada.

Kubernetes é uma plataforma de orquestração de containers: resolve o problema de rodar, escalar e manter disponíveis centenas ou milhares de containers de forma automatizada, algo inviável de gerenciar manualmente com comandos Docker isolados.

11.1 Conceitos Fundamentais

Termo	Definição
Cluster	Conjunto de máquinas (nodes) gerenciadas como uma unidade
Node	Uma máquina (física ou virtual) que faz parte do cluster
Pod	Menor unidade implantável; um ou mais containers que compartilham rede e armazenamento
Deployment	Define o estado desejado de um conjunto de pods (quantas réplicas, qual imagem)
Service	Ponto de acesso estável para um conjunto de pods, mesmo que eles mudem de IP
Namespace	Espaço lógico para isolar recursos dentro do mesmo cluster
ConfigMap	Armazena configuração não sensível, desacoplada da imagem
Secret	Armazena dados sensíveis (senhas, tokens), com controle de acesso mais restrito
Ingress	Gerencia acesso externo HTTP/HTTPS aos services do cluster

11.2 Scaling, Scheduling e Health Checks

O **scheduler** do Kubernetes decide em qual node cada pod deve rodar, com base em recursos disponíveis. O cluster pode fazer **scaling** automático do número de pods (*Horizontal Pod Autoscaler*) conforme a carga observada. **Health checks** (*liveness* e *readiness probes*) permitem que o Kubernetes detecte pods travados ou ainda não prontos, substituindo-os automaticamente quando necessário — o mesmo princípio do health check de um Load Balancer, aplicado no nível de container.

IX

Infrastructure as Code

CAPÍTULO 12

Terraform

12.1 Por que IaC é importante

Imagine precisar provisionar manualmente 20 VMs, 10 redes, 5 load balancers e 30 regras de firewall, uma por uma, clicando em um console. Além de lento, o processo é propenso a erro humano e impossível de replicar de forma idêntica em outro ambiente.

Infrastructure as Code (IaC) resolve isso definindo toda a infraestrutura em arquivos de código, versionados como qualquer código de aplicação:

- **Replicabilidade**: o mesmo código recria o ambiente idêntico em qualquer lugar.
- **Versionamento**: com Git, é possível rastrear cada mudança na infraestrutura ao longo do tempo.
- **Redução de erro humano**: o mesmo processo automatizado roda de forma consistente toda vez.

12.2 Conceitos do Terraform

Termo	Definição
Provider	Plugin que conecta o Terraform a uma plataforma (AWS, Azure, etc.)
Resource	Um componente de infraestrutura a ser criado (uma VM, uma rede)
Variable	Parâmetro de entrada, reutilizável entre execuções
Output	Valor exposto após a criação (ex: IP público gerado)
State	Arquivo que registra o estado atual real da infraestrutura gerenciada
Module	Conjunto reutilizável de recursos, agrupados como um bloco

Listing 12.1: Exemplo simplificado de configuração Terraform

```
provider "aws" {
  region = "sa-east-1"
}

variable "instance_type" {
  default = "t3.micro"
}

resource "aws_instance" "servidor_web" {
  ami           = "ami-0123456789abcdef"
  instance_type = var.instance_type

  tags = {
```

```
    Name = "servidor-web"
  }
}

output "ip_publico" {
  value = aws_instance.servidor_web.public_ip
}
```

Listing 12.2: Fluxo básico de comandos

```
terraform init      # baixa providers e inicializa o projeto
terraform plan      # mostra o que sera criado/alterado/destruido
terraform apply     # aplica as mudancas de fato
terraform destroy   # remove toda a infraestrutura criada
```

12.3 Drift e Organização em Módulos

Drift acontece quando o estado real da infraestrutura diverge do que está definido no código — por exemplo, alguém altera algo manualmente pelo console, fora do fluxo do Terraform. O comando `terraform plan` detecta esse desalinhamento antes de aplicar novas mudanças.

Em projetos maiores, a infraestrutura costuma ser organizada em **módulos** separados por responsabilidade:

Listing 12.3: Organização típica de um projeto Terraform

```
terraform/
  network/       -> VPC, subnets, route tables
  compute/       -> instancias EC2
  storage/       -> volumes, buckets
  load_balancer/ -> load balancers e target groups
  firewall/      -> security groups
```

X

Scripting e Automação

CAPÍTULO 13

Bash

13.1 Fundamentos

Listing 13.1: Variáveis, condicionais e loops

```
# Variaveis
NOME="servidor-01"
echo $NOME

# Condicional
if [ "$STATUS" == "ativo" ]; then
  echo "Servico rodando"
else
  echo "Servico parado"
fi

# Loop
for arquivo in $(ls /var/log); do
  echo "Encontrado: $arquivo"
done
```

13.2 Redirecionamento e Pipes

Listing 13.2: Redirecionamento e encadeamento de comandos

```
comando > arquivo.txt      # sobrescreve o arquivo com a saida
comando >> arquivo.txt     # adiciona a saida ao final do arquivo
comando1 | comando2        # envia a saida do comando1 como entrada do
    comando2
```

13.3 Ferramentas de Texto: grep, awk, sed, cut, sort, xargs

Listing 13.3: Manipulação de texto e logs

```
grep -i "error" app.log          # filtra linhas contendo "error" (case-
    insensitive)
awk '{print $1}' access.log      # imprime a primeira coluna de cada
    linha
sed 's/erro/ERRO/g' log.txt      # substitui ocorrencias de texto
cut -d',' -f2 dados.csv          # extrai a segunda coluna de um CSV
sort -n numeros.txt              # ordena numericamente
xargs -I{} rm {}                 # aplica um comando a cada linha
    recebida
```

13.4 Cron

O **cron** agenda execução periódica de scripts, sem intervenção manual.

Listing 13.4: Exemplo de crontab: executar script a cada 5 minutos

```
*/5 * * * * /home/thiago/scripts/verifica_servico.sh
```

13.5 Script Real: Verificação de Serviço com Alerta

Listing 13.5: Verificar se um serviço está ativo, com log

```
#!/bin/bash

SERVICO="nginx"
LOG="monitor.log"

if systemctl is-active --quiet "$SERVICO"; then
    echo "OK: o servico $SERVICO esta rodando normalmente."
else
    echo "$(date '+%d/%m/%Y %H:%M:%S') - ALERTA: $SERVICO nao esta ativo!
       " >> "$LOG"
    echo "ALERTA registrado no log."
fi
```

Executando: `chmod +x verifica_servico.sh` para tornar o arquivo executável, seguido de `./verifica_servico`
Esse padrão — verificar condição, agir, registrar em log com data/hora — é a base de praticamente qualquer script de monitoramento simples em ambiente de sustentação.

CAPÍTULO 14

Python para Infraestrutura

14.1 Bibliotecas Relevantes

Biblioteca	Uso
`psutil`	Coleta de métricas do sistema: CPU, memória, disco, processos
`requests`	Chamadas HTTP para APIs externas ou health checks de serviços
`boto3`	SDK oficial da AWS para Python — automatiza criação/consulta de recursos

14.2 Script Real: Monitoramento de Disco

Listing 14.1: Verificação de espaço em disco com alerta e log

```python
import shutil
from datetime import datetime

def verificar_disco():
    try:
        total, usado, livre = shutil.disk_usage("/")
        porcentagem_usado = (usado / total) * 100

        if porcentagem_usado > 90:
            agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
            mensagem = f"{agora} - ALERTA: disco em {porcentagem_usado:.1
                f}% de uso\n"
            print(mensagem)
            with open("monitor.log", "a") as arquivo:
                arquivo.write(mensagem)
        else:
            print(f"Disco OK, uso atual: {porcentagem_usado:.1f}%")

    except Exception as e:
        print("Nao foi possivel verificar o status do disco:", e)

verificar_disco()
```

ATENÇÃO

Em automação de infraestrutura, um script sem tratamento de erro (`try/except`) é uma falha grave, não apenas um detalhe estético: sem ele, um problema inesperado pode interromper a execução silenciosamente, comprometendo toda a automação sem gerar alerta nenhum.

14.3 Extensão Natural: psutil e requests

O mesmo padrão do script de disco se estende naturalmente a CPU e memória usando `psutil`:

Listing 14.2: Padrão equivalente para CPU (usando psutil)

```
import psutil
from datetime import datetime

def verificar_cpu(limite=90):
    uso = psutil.cpu_percent(interval=1)
    if uso > limite:
        agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
        print(f"{agora} - ALERTA: CPU em {uso}%")
    else:
        print(f"CPU OK: {uso}%")

verificar_cpu()
```

Listing 14.3: Verificando se um serviço web está no ar

```
import requests

def verificar_servico(url):
    try:
        resposta = requests.get(url, timeout=5)
        if resposta.status_code == 200:
            print("Servico no ar.")
        else:
            print(f"Servico com problema, codigo: {resposta.status_code}"
                )
    except requests.exceptions.RequestException:
        print("Servico inacessivel.")

verificar_servico("https://exemplo.com")
```

XI

Observabilidade

CAPÍTULO 15

Monitoramento e Observabilidade

15.1 Métricas, Logs, Traces e Eventos

Pilar	O que representa
Métricas	Valores numéricos ao longo do tempo (CPU %, memória, latência)
Logs	Registros textuais e discretos de eventos específicos
Traces	Rastreamento do caminho de uma requisição através de múltiplos serviços
Eventos	Marcos pontuais relevantes (deploy, reinicialização, alteração de configuração)
Alertas	Notificações disparadas quando métricas cruzam limites definidos

15.2 Monitoramento vs. Observabilidade vs. Troubleshooting

- **Monitoramento**: responde "o sistema está saudável agora?" — baseado em métricas e alertas predefinidos.
- **Observabilidade**: vai além — permite investigar "por que isso está acontecendo?" mesmo para problemas nunca previstos antes, cruzando métricas, logs e traces.
- **Troubleshooting**: o processo ativo de investigação humana, usando as ferramentas de observabilidade disponíveis, para chegar à causa raiz de um problema específico.

PERGUNTA DE REVISÃO

Por que uma empresa investiria em observabilidade se já tem monitoramento básico funcionando?
Porque monitoramento só avisa sobre problemas previstos previamente (ex: "CPU acima de 90%"). Observabilidade permite investigar problemas nunca antes vistos, correlacionando múltiplas fontes de dado — essencial à medida que sistemas crescem em complexidade e falhas deixam de ser sempre óbvias.

XII

Sustentação e Troubleshooting

CAPÍTULO 16

Mentalidade de Sustentação

Esta é, talvez, a parte mais importante deste handbook. Conhecimento técnico isolado — saber o que é uma subnet, o que é um Security Group — é necessário, mas não suficiente. O que realmente diferencia um bom profissional de sustentação é a **metodologia** de investigação diante de um incidente real.

16.1 Metodologia de Investigação



Etapa	Pergunta-guia
Sintoma	O que está sendo observado/reportado, exatamente?
Hipótese	Quais são as causas prováveis, dado o sintoma?
Coleta de evidências	Quais dados (logs, métricas, comandos) confirmam ou descartam cada hipótese?
Isolamento	O problema está na aplicação, no sistema operacional, na rede, ou em dependência externa?
Diagnóstico	Qual é, de fato, a causa raiz confirmada?
Correção	Qual ação resolve o problema imediato?
Validação	O sintoma original desapareceu de verdade, e nada mais quebrou?
Prevenção	O que evita que isso se repita?

ATENÇÃO

Pular direto para “correção” sem antes isolar a causa raiz é o erro mais comum em situação de pressão. Um sintoma como “aplicação lenta” pode ter dezenas de causas possíveis — aplicar uma solução antes de confirmar a hipótese certa desperdiça tempo precioso durante um incidente e pode mascarar o problema real.

16.2 Cenário 1: Servidor Inacessível

Sintoma: não é possível acessar o servidor

Possíveis causas: problema de DNS; regra de firewall/Security Group bloqueando; serviço parado; porta errada; processo travado consumindo todos os recursos; problema de rede entre cliente e servidor.
Comandos:

```
ping servidor.exemplo.com          # a rede basica ate o host responde?
dig servidor.exemplo.com           # o DNS esta resolvendo corretamente?
telnet servidor.exemplo.com 443    # a porta especifica esta acessivel?
systemctl status nome-servico      # o servico esta ativo no host?
top                                # ha algum processo consumindo tudo?
tail -f /var/log/nome-servico.log  # ha erro recente no log do servico?
```

Interpretação: se o `ping` funciona mas a porta não responde, o problema provavelmente está no serviço ou no firewall — não na rede em si. Se nem o `ping` funciona, o problema é mais fundamental (rede, DNS, ou o host está de fato fora do ar).

16.3 Cenário 2: Aplicação Lenta

Sintoma: usuários reportam lentidão

Possíveis causas: CPU ou memória no limite; I/O de disco saturado; latência de rede; consulta lenta no banco de dados; dependência externa lenta (API de terceiro).
Comandos:

```
top / htop                 # identifica processo consumindo CPU/memoria
free -h                    # verifica pressao de memoria (uso de swap)
iostat -x 1                # mostra utilizacao de I/O de disco
netstat -tulnp             # conexoes ativas, possiveis gargalos de rede
tail -f /var/log/app.log   # erros ou lentidao reportada nos logs da app
```

Interpretação: priorize o recurso mais próximo do limite (CPU vs memória vs disco vs rede) — geralmente apenas um deles é o gargalo real, e otimizar os outros não resolve nada até que o gargalo verdadeiro seja tratado.

16.4 Cenário 3: Aplicação Não Consegue Acessar o Banco

Sintoma: erro de conexão com banco de dados

Possíveis causas: DNS do banco não resolve; rota de rede incorreta entre subnets; porta bloqueada pelo Security Group; credenciais incorretas ou expiradas; serviço do banco parado.
Comandos:

```
dig banco.interno.exemplo.com      # o nome do banco resolve
    corretamente?
telnet banco.interno.exemplo.com 5432    # a porta do banco esta
    acessivel?
psql -h host -U usuario -d banco  # testar conexao/credenciais
    diretamente
```

Interpretação: se a porta não abre via `telnet`, o problema é de rede/Security Group, não de credencial — credenciais só importam depois que a conexão de rede já foi estabelecida com sucesso.

16.5 Cenário 4: Servidor Sem Espaço

Sintoma: erro de "no space left on device"

Possíveis causas: logs crescendo sem rotação; arquivos temporários não limpos; volumes/imagens Docker órfãs acúmulando espaço; crescimento legítimo de dados sem planejamento de capacidade.
Comandos:

```
df -h                              # visao geral de uso por particao
du -sh /var/log/*                  # quais diretorios de log ocupam mais
    espaco
docker system df                   # espaco ocupado por imagens/volumes
    Docker
docker system prune -a             # remove imagens e volumes nao
    utilizados
```

Interpretação: `df -h` mostra o sintoma (qual partição está cheia); `du -sh` localiza a causa (quais diretórios específicos consomem o espaço). Nunca limpe algo sem antes confirmar o que é, especialmente em produção.

XIII

Projetos e Laboratórios

CAPÍTULO 17

Laboratórios Realizados

17.1 Laboratório 1 — Script de Monitoramento de Disco (Python)

Objetivo
Detectar automaticamente quando o uso de disco ultrapassa um limite crítico, registrando um alerta com data e hora.

Arquitetura
Script Python standalone, executado manualmente ou via `cron`, escrevendo em um arquivo de log local.

Pré-requisitos
Python 3 instalado; nenhuma biblioteca externa (usa apenas `shutil` e `datetime`, da biblioteca padrão).

Implementação
Função única que consulta `shutil.disk_usage("/")`, calcula o percentual usado, e condicionalmente grava um alerta em log.

Comandos
`python3 disco_monitor.py`

Explicação
`shutil.disk_usage` retorna uma tupla nomeada com total, usado e livre em bytes; a proporção entre usado e total independe da unidade.

Testes
Validado com o limite normal (90%) e temporariamente reduzido para 1% para forçar a geração do alerta e confirmar a escrita correta no log.

Possíveis erros
Falha de permissão para escrever o arquivo de log; caminho de disco inexistente no ambiente onde o script roda.

Troubleshooting
Bloco `try/except` captura qualquer exceção e imprime mensagem descritiva, evitando que o script quebre silenciosamente.

O que aprendi
A importância de tratar erro em automação de infraestrutura, e como formatar corretamente data/hora para registros de log legíveis (`strftime`).

17.2 Laboratório 2 — Script de Verificação de Serviço (Bash)

Objetivo
Verificar se um serviço gerenciado pelo systemd está ativo, registrando alerta em log quando não estiver.

Arquitetura
Script Bash standalone, usando `systemctl is-active` como fonte de verdade do status do serviço.

Pré-requisitos
Sistema Linux com `systemd`; permissão de execução do script.

Implementação
Condicional `if systemctl is-active -quiet $SERVICO` decide entre mensagem de sucesso ou gravação de alerta com timestamp.

Comandos
```
chmod +x verifica_servico.sh && ./verifica_servico.sh
```

Explicação
A flag `-quiet` suprime saída do comando, usando apenas o código de saída (0 = ativo) na condicional.

Testes
Executado tanto com um serviço ativo quanto com um serviço inexistente/parado, confirmando os dois caminhos do `if/else`.

Possíveis erros
Nome do serviço incorreto; falta de permissão de execução do script.

Troubleshooting
Rodar `systemctl status nome-servico` manualmente para confirmar o nome exato do serviço antes de automatizar a verificação.

O que aprendi
A diferença entre escrever código Bash direto no terminal (que quebra com estruturas de múltiplas linhas) e salvar como arquivo `.sh` executado de uma vez.

17.3 Laboratório 3 — Arquitetura Conceitual de VPC

Objetivo
Consolidar o entendimento de como organizar uma VPC em camadas (*tiers*) de segurança, com base em material estudado sobre a arquitetura da Locaweb Cloud.

Arquitetura
VPC única dividida em três subnets: *tier web* (pública), *tier app* (privada) e *tier DB* (privada), cada uma com seu bloco CIDR próprio.

Pré-requisitos
Entendimento prévio de CIDR, Internet Gateway, NAT Gateway e Security Groups.

Implementação (conceitual)
Internet Gateway associado à subnet pública; NAT Gateway compartilhado atendendo as subnets privadas; Security Groups em camada, liberando apenas o tráfego estritamente necessário entre tiers.

Comandos
N/A — exercício conceitual de desenho de arquitetura, não provisionado em ambiente real neste momento.

Explicação
A separação em tiers segue o princípio de menor exposição: apenas o que precisa ser público (o balanceador de carga) fica exposto; aplicação e banco ficam isolados.

Testes
N/A.

Possíveis erros
Esquecer de configurar a rota da subnet privada para o NAT Gateway (resultando em recursos privados sem saída à internet).

Troubleshooting
Validar as *route tables* de cada subnet individualmente confirma se a rota esperada (Internet Gateway ou NAT Gateway) está de fato associada.

O que aprendi
Que Security Group e subnet resolvem problemas diferentes e complementares — confundir os dois é um erro conceitual comum.

XIV

Arquitetura de Ponta a Ponta

CAPÍTULO 18

Conectando Todos os Conceitos

Uma requisição real, do clique do usuário até o dado sendo retornado, atravessa praticamente todas as camadas estudadas neste handbook:

Diagrama — Fluxo completo de uma requisição

```
              USUARIO
                 |
                DNS            (resolve o dominio para um IP)
                 |
         LOAD BALANCER         (subnet publica, distribui trafego)
                 |
           APLICACAO           (container/EC2, subnet privada)
                 |
            CONTAINER          (isolamento de processo, kernel do host)
                 |
             DATABASE          (subnet privada, isolada por Security Group)
                 |
             STORAGE           (persistencia dos dados)
```

Cada camada carrega os conceitos das partes anteriores: o **DNS** resolve o nome do serviço; o **Load Balancer** decide para qual instância a requisição vai, com base em **health checks**; a **aplicação** roda sobre um **sistema operacional Linux**, possivelmente dentro de um **container**, comunicando-se com o **banco de dados** através de regras definidas em **Security Groups**; e tudo isso é provisionado de forma consistente via **Infrastructure as Code**, monitorado por ferramentas de **observabilidade**, e mantido operacional pela **mentalidade de sustentação** descrita na Parte XIV.

PERGUNTA DE REVISÃO

Se o usuário reporta que a página não carrega, em qual ordem você verificaria as camadas desse diagrama?
De fora para dentro: primeiro DNS (o nome resolve?), depois Load Balancer/rede (a porta responde?), depois aplicação (o processo está rodando, sem erro nos logs?), só então banco de dados. Isso evita perder tempo investigando o banco quando o problema real é, por exemplo, DNS não resolvendo.

XV

Entrevista de Cloud Engineering

CAPÍTULO 19

Perguntas e Respostas por Nível

As respostas a seguir buscam refletir compreensão real do conceito — explicando o raciocínio por trás, não apenas repetindo uma definição decorada.

19.1 Nível Básico — Fundamentos

O que acontece quando você digita um domínio no navegador?

Resposta ideal: o navegador primeiro resolve o domínio via DNS, seguindo a hierarquia até obter o IP do servidor autoritativo. Com o IP em mãos, estabelece uma conexão TCP (handshake de 3 vias) e, se for HTTPS, negocia TLS para criptografar a comunicação. Depois, envia a requisição HTTP, que pode passar por um Load Balancer antes de chegar ao servidor de aplicação, que processa e retorna a resposta.
O que se avalia: visão de ponta a ponta, conectando DNS, TCP, TLS e HTTP como camadas de um mesmo fluxo.

O que é DNS?

Resposta ideal: sistema que traduz nomes de domínio legíveis por humanos em endereços IP usados pelas máquinas para se comunicar, seguindo uma hierarquia de servidores (root, TLD, autoritativo).
O que se avalia: clareza na explicação e conhecimento do fluxo de resolução, não apenas a definição superficial.

Qual a diferença entre TCP e UDP?

Resposta ideal: TCP é orientado a conexão, com handshake e garantia de entrega ordenada — usado onde confiabilidade importa mais que velocidade (HTTP, SSH). UDP não estabelece conexão prévia, é mais rápido mas sem garantias — usado em streaming e chamadas de voz, onde perder um pacote é preferível a atrasar.
O que se avalia: capacidade de explicar o *porquê* de cada protocolo existir, com exemplo prático de uso.

O que é CIDR?

Resposta ideal: notação (`IP/prefixo`) usada para representar um bloco de endereços IP, onde o número após a barra indica quantos bits definem a parte de rede — o restante define quantos endereços estão disponíveis dentro daquele bloco.
O que se avalia: entendimento prático, incluindo capacidade de calcular ou estimar quantos endereços um bloco específico oferece.

O que é uma subnet, e qual a diferença entre pública e privada?

Resposta ideal: subnet é uma subdivisão lógica de uma rede maior (VPC), com seu próprio bloco CIDR. Subnet pública tem rota para um Internet Gateway (entrada e saída de tráfego); subnet privada não recebe conexão externa diretamente, mas pode ter saída via NAT Gateway.
O que se avalia: a nuance de que privada **não** significa ausência total de internet, apenas ausência de conexão de entrada.

19.2 Nível Intermediário — Troubleshooting

O que é NAT Gateway?

Resposta ideal: recurso que permite que instâncias em subnets privadas iniciem conexões de saída à internet (para atualizações, chamadas de API externas), sem expô-las a conexões de entrada vindas de fora. Um único NAT Gateway pode atender múltiplas instâncias privadas.
O que se avalia: entendimento do fluxo assimétrico (saída sim, entrada não), e não apenas "ele dá internet pra rede privada".

O que é Security Group?

Resposta ideal: firewall associado diretamente a um recurso (não à subnet como um todo), definindo quais conexões de entrada e saída são permitidas. Diferente de subnet, que define *onde* o recurso está e quais rotas possui — Security Group define *quem* pode se conectar a ele.
O que se avalia: clareza na distinção entre os dois conceitos, que costumam ser confundidos.

Como você investigaria um servidor inacessível?

Resposta ideal: seguir uma metodologia estruturada — primeiro confirmar se é problema de DNS (`dig`), depois de rede/porta (`ping`, `telnet`), depois se o serviço está de fato ativo no host (`systemctl status`), e por último revisar logs em busca de erro específico. Isolar camada por camada evita presumir a causa antes de confirmar evidência.
O que se avalia: metodologia de investigação, não apenas conhecimento isolado de comandos.

Como você investigaria uma aplicação lenta?

Resposta ideal: verificar CPU, memória, I/O de disco e rede, identificando qual recurso está próximo do limite — geralmente há um único gargalo real. Cruzar isso com logs da aplicação para ver se há erro ou lentidão explícita reportada, e considerar dependências externas (banco de dados, APIs de terceiros) como possível origem do atraso.
O que se avalia: capacidade de priorizar hipóteses em vez de investigar tudo aleatoriamente.

Container é uma VM?

Resposta ideal: não. VM virtualiza hardware completo, com sistema operacional próprio isolado; container compartilha o kernel do sistema operacional hospedeiro, isolando apenas processos — por isso é muito mais leve e rápido de iniciar, porém com isolamento de segurança mais fraco que uma VM.
O que se avalia: precisão técnica na diferença, evitando a confusão comum de tratar os dois como sinônimos.

19.3 Nível Avançado — Arquitetura e Incidêntes

O que é Infrastructure as Code, e por que é melhor que configuração manual?

Resposta ideal: prática de definir infraestrutura em código (via ferramentas como Terraform), trazendo replicabilidade (recriar ambientes idênticos), versionamento (rastrear mudanças via Git) e redução de erro humano — especialmente relevante quando o número de recursos gerenciados cresce (dezenas de VMs, redes e regras).
O que se avalia: capacidade de ir além de "é mais rápido", citando os três benefícios estruturais.

Como você investigaria um incidente em produção?

Resposta ideal: seguir a metodologia sintoma → hipótese → coleta de evidências → isolamento → diagnóstico → correção → validação → prevenção. Priorizar restaurar o serviço (mitigação) antes de necessariamente entender 100% da causa raiz, se o impacto for crítico — e só depois investigar a fundo para prevenir recorrência.
O que se avalia: equilíbrio entre agir rápido sob pressão e não pular etapas de investigação de forma descuidada.

O que é Kubernetes, e quando ele se justifica em relação a rodar containers "soltos" com Docker?

Resposta ideal: Kubernetes é uma plataforma de orquestração que automatiza a execução, escalonamento e recuperação de containers em escala. Faz sentido quando a quantidade de containers e a necessidade de alta disponibilidade tornam inviável o gerenciamento manual — para poucos containers em ambiente simples, a complexidade adicional do Kubernetes pode não se justificar.
O que se avalia: senso crítico sobre quando uma ferramenta complexa é justificada, evitando a resposta ingênua de "sempre use Kubernetes".

XVI

Glossário

Glossário Técnico

Termo	Definição e contexto
AMI	Imagem base usada para lançar uma instância EC2 na AWS.
Availability Zone	Datacenter fisicamente isolado dentro de uma região de nuvem.
CIDR	Notação de bloco de endereços IP (`IP/prefixo`).
CloudWatch	Serviço de monitoramento e logs da AWS.
Container	Processo isolado, empacotado com suas dependências, compartilhando o kernel do host.
Cron	Agendador de tarefas periódicas no Linux.
Deployment (K8s)	Define o estado desejado de um conjunto de pods no Kubernetes.
DNS	Sistema que traduz nomes de domínio em endereços IP.
Drift (Terraform)	Divergência entre o estado real da infraestrutura e o definido em código.
EBS	Armazenamento em bloco persistente da AWS, separado do ciclo de vida da instância.
EC2	Serviço de máquinas virtuais sob demanda da AWS.
Elasticidade	Capacidade de um ambiente se ajustar automaticamente à demanda.
Fault Tolerance	Capacidade do sistema continuar operando mesmo com falha de um componente.
Firewall	Filtro de tráfego de rede baseado em regras.
Gateway	Ponto de saída de uma rede local para outras redes.
Handshake (TCP)	Processo de três vias (SYN, SYN-ACK, ACK) que estabelece uma conexão TCP.
Health Check	Verificação periódica usada por Load Balancers/Kubernetes para detectar falhas.
Hypervisor	Software que gerencia máquinas virtuais sobre um hardware físico.
IaaS	Modelo de nuvem onde o provedor entrega infraestrutura bruta (VMs, rede).
IaC	Infrastructure as Code — definir infraestrutura via código.
IAM	Serviço de gerenciamento de identidade e permissões da AWS.
Internet Gateway	Componente que conecta uma VPC à internet.
IOPS	Operações de entrada/saída por segundo que um armazenamento processa.
Kernel	Núcleo do sistema operacional, compartilhado entre containers no mesmo host.
Load Balancer	Distribui requisições entre múltiplas instâncias.
NAT Gateway	Permite saída à internet para recursos em subnet privada, sem expor entrada.
Namespace (K8s)	Espaço lógico de isolamento dentro de um cluster Kubernetes.
Node (K8s)	Máquina que faz parte de um cluster Kubernetes.
Observabilidade	Capacidade de investigar problemas cruzando métricas, logs e traces.

Termo	Definição e contexto
PaaS	Modelo de nuvem onde o provedor entrega plataforma pronta para rodar código.
Pod (K8s)	Menor unidade implantável no Kubernetes, um ou mais containers.
Policy (IAM)	Documento que define quais ações são permitidas ou negadas.
Provider (Terraform)	Plugin que conecta o Terraform a uma plataforma específica.
Region	Área geográfica onde um provedor de nuvem mantém infraestrutura.
Role (IAM)	Identidade temporária assumível por usuários ou serviços.
Route Table	Define para onde o tráfego de uma subnet é direcionado.
SaaS	Modelo de nuvem onde o provedor entrega software pronto para uso final.
Scaling Horizontal	Adicionar mais máquinas para suportar mais carga.
Scaling Vertical	Aumentar os recursos de uma única máquina.
Security Group	Firewall associado diretamente a um recurso de nuvem.
Service (K8s)	Ponto de acesso estável a um conjunto de pods.
Shared Responsibility	Modelo que define os limites entre segurança do provedor e do cliente.
Snapshot	Ponto de restauração de um disco/VM em determinado momento.
State (Terraform)	Arquivo que registra o estado real da infraestrutura gerenciada.
Subnet	Subdivisão lógica de uma rede maior (VPC).
Systemd	Sistema de inicialização e gerenciamento de serviços do Linux moderno.
TCP	Protocolo de transporte confiável, orientado a conexão.
Template (Virtualização)	Imagem base usada para criar novas VMs rapidamente.
TLS	Protocolo de criptografia usado para proteger comunicações (base do HTTPS).
TTL (DNS)	Tempo que uma resposta DNS permanece em cache.
UDP	Protocolo de transporte sem conexão, mais rápido e sem garantia de entrega.
VPC	Rede virtual privada e isolada dentro de um provedor de nuvem.

— Fim do Handbook —